

Graduate Algorithms

What Your Midsem Could Look Like...

20 September 2026

Look, there are more important problems in the world, but this is something that I can do that is helpful and I enjoy doing. So it's how I choose to give back.

— Craig Campanella¹

THIS IS NOT A MOCK! THIS IS NOT A PREDICTION! IF ANYTHING, IT IS FOR PREPARATION PURPOSE ONLY.

QUESTIONS ARE SOMEWHAT HARDER THAN WHAT CAN BE EXPECTED BUT THEN AGAIN, 1. BETTER SAFE THAN SORRY 2. YOU NEED TO ATTEMPT ONLY HALF THE PAPER.

This exam has **6 questions** for a **total of 600 marks**, of which you can score **at most 300 marks**. You may answer any subset of questions or parts of questions. All answers will be evaluated. Go through all the questions once before you start writing your answers.

You may use the following functions, and assume in your analysis that they work correctly with the stated worst-case running times:

- `LENGTH(A)`: returns the length n of array A , and runs in $O(n)$ time
- `MERGESORT(A)`: returns the version of integer array A which is sorted in non-decreasing order, and runs in $O(n \log n)$ time when A has n elements;
- `BINARYSEARCH(A, v)`: returns the index of integer element v in a sorted array A , or `NIL` if v is not present in A . Runs in $O(\log n)$ time when A has n elements.
- `DIJKSTRA(G, s, t)`: returns the shortest distance between s and t in a weighted (directed) graph G in $O((m + n) \log(n))$ where G has n vertices and m edges.

You may freely invoke functions that you have written as part of a different answer in the same answer sheet. You do not have to use loop invariants while proving the correctness of algorithms; but you must correctly explain why each loop (if there are some) does what you expect it to do.

Clearly describe the meaning of any syntax that you use. E.g., if you use the notation $A[i : j]$ to denote a sub-array, clearly explain what you mean by this. And similarly for any other notation. You may assume the following when analyzing the running time of algorithms: (i) comparing a pair of numbers takes constant time; (ii) reading/writing an array element using its index takes constant time, and, (iii) creating a new array, and returning an array, each takes time linear in the length of the array. You must clearly state any other assumption that you make.

You may use the Master Theorem for solving recurrences (except when the question says otherwise), but you must properly state the version of the theorem that you use, before applying it.

Unstated assumptions and lack of clarity in solutions can and will be used against you during evaluation. You may freely refer to statements from the lectures in your arguments. You don't need to reprove these unless the question explicitly asks you to, but you must be precise.

¹A New Yorker who fixes basketball nets in public parks in free time

1. Unroll each of the following recurrences to come up with an estimate $f(n)$ that satisfies $T(n) = \Theta(f(n))$.

In each case, verify your estimate by induction. Note that this involves verifying *two* asymptotic bounds for each part, for the *same* function f .

In each case, you may assume bounds of the form $T(n') \leq c'$ and $T(n'') \geq c''$ where n', n'', c', c'' are all fixed constants of your choice. That is, you may assume constant upper and lower bounds for inputs of up to some constant size.

You will *not* get the credit for either part if you use the Master Theorem (or some such) to get at the estimate.

(a) $T(n) = 6T\left(\frac{n}{9}\right) + 69\log(n)$ (25 Marks)

(b) $T(n) = 2T(\sqrt{n}) + O(n \log^2 n)$ (25 Marks)

2. Nick Fury is putting together a team for an important mission. Obviously, he would like the mission to be a success.

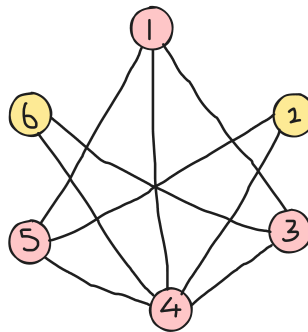
Furthermore, Fury is quite certain that, for the team to work well, every selected hero has good chemistry with every other selected hero. He is currently trying to come up with a list of heroes he would like to recruit.

Fury has n potential recruits, where n is divisible by 3. Most heroes have a good chemistry.

Fury also recalls that during Endgame (the Thanos incident) exactly $2\frac{n}{3}$ of these heroes were present and everyone had good chemistry. Unfortunately, his memory of it is rather hazy. In particular, he has no idea which of the heroes were involved.

Fury does not need to assemble a massive team, it's a simpler mission but he would like to recruit at least $\frac{n}{3}$ heroes.

For example, see below. The red is the team that had assembled for the Thanos incident while for this mission, any two people who know each other would suffice (hence, any edge).



- (a) As SHIELD's Algorithmic head, you are requested to provide a pseudo-code implementing a function `mission_team(G)` that takes a simple graph $G = (V, E)$ where the vertices V represent the heroes and the edges E represent heroes who have good chemistry together; provides a team $T \subseteq V$ every hero in the team has good chemistry and $|T| \geq \frac{n}{3}$ that runs in $O(|V|^2)$.

(70 marks)

Note: $|V| = n$ and $\frac{2n(\frac{2}{3}n-1)}{2} \leq |E| \leq \frac{n(n-1)}{2}$. Also, formally, $\forall u, v \in T : uv \in E$.

- (b) Argue the correctness of your algorithm.

(50 marks)

3. CWT's physics lab has installed n machines for processing matter and anti-matter. The machines are arranged in a row and machine i has capacity $C[i]$.

An experiment is safe if:

- A contiguous set of machines are operational at full capacity.
- The sum amount of matter and anti-matter being processed is equal.

(a) Write the complete pseudo-code for a recursive algorithm `experiments(C)` that given the arrangement and capacities of the machines, returns the number of safe experiments CWT's physics lab can perform. **(40 Marks)**

For example, `experiments([1,1,1,1])` should output **12** where $[1+, 2-]$, $[1-, 2+]$, $[2+, 3-]$, $[2-, 3+]$, $[3+, 4-]$, $[3-, 4+]$, $[1+, 2+, 3-, 4-]$, $[1+, 2-, 3+, 4-]$, $[1+, 2-, 3-, 4+]$, $[1-, 2+, 3+, 4-]$, $[1-, 2+, 3-, 4+]$ and $[1-, 2-, 3+, 4+]$ are safe experiments ($i+$ represents machine i processing matter and $i-$ represents machine i processing anti-matter).

(b) In terms of n and $m = \text{sum}(C)$, what is the runtime of your recursive solution? **(20 Marks)**

(c) Write pseudo-code for the memoized version of (a) without using any libraries or decorators or language features. **(40 Marks)**

(d) What is the runtime of your memoized solution? **(20 Marks)**

4. Wingull has been given a special mission! Starting from his home in Slateport, Wingull must collect a Tofu Sandwich and return home as quickly as possible.

He has a map of the n islands in the area, some set T of which have Tofu Sandwiches. There are m one-way navigable sea routes between the islands. Wingull can swim over a route from island i to j in $t(i, j)$ minutes, where $t(i, j)$ is a positive integer. He can also fly over any route in one minute. After flying, Wingull must rest his wings for one hour and seven minutes before flying again. During this time, he can swim and/or wait.

(a) Write pseudo-code implementing `best_route(G, t, s, T)` which given a weighted directed graph $G = (V, E)$ with the weights t where vertices represent the islands, $s \in V$ is Slateport, $T \subseteq V$ represents the islands with Tofu Sandwiches and edges represent the one way sea routes; returns the minimum time, starting from Slateport to collect a Tofu Sandwich and return home in $O((n + m) \log(n))$ time. **(70 Marks)**

For example, see below. Let the green squares be the islands with Tofu Sandwiches.

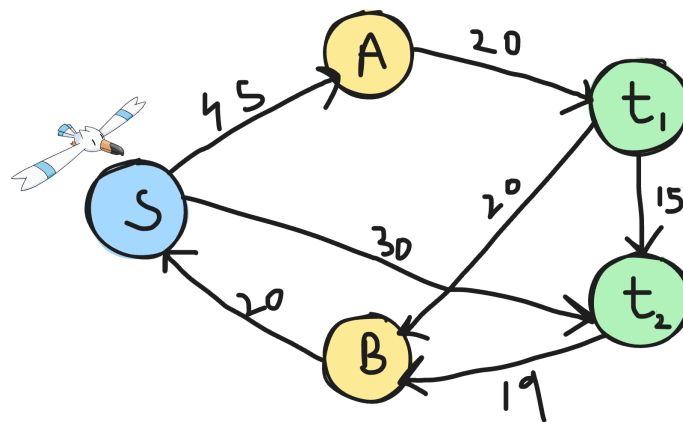


Figure 2: Notice $S \xrightarrow{\text{fly}} t_1 \xrightarrow{\text{swim}} B \xrightarrow{\text{swim}} S$ takes 40 mins. No other set of moves is better.

(b) Argue that your algorithm runs in $O((n + m) \log(n))$ time indeed. **(15 Marks)**

(c) Argue the correctness of your algorithm. **(15 Marks)**

Hint 1: You probably only know one algorithm that finds the shortest path between source and target on a directed graph.

Hint 2: $O(42n) = O(n)$.

5. CMI's guitar club has n guitars of length $L[1, 2, \dots, n]$ along with n cases of size $C[1, 2, \dots, n]$. A guitar i fits the case j if $l_i \leq c_j$.

(a) Write pseudocode implementing `can_fit(L, C)` which given the lengths of the guitars and sizes of the cases; returns True if we can fit all the guitars into the cases and false otherwise that runs in $O(n \log n)$ time. **(35 marks)**

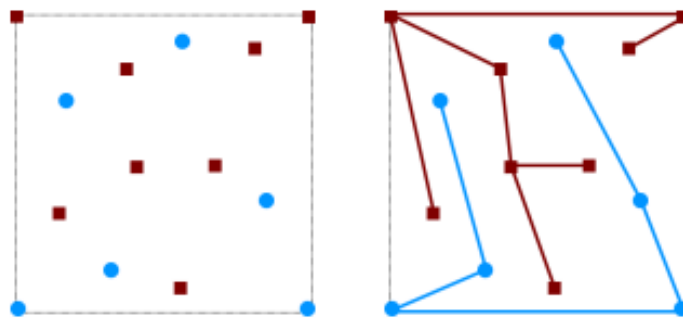
(b) Argue the correctness of your implementation. **(15 marks)**

6. A number of red points and blue points are drawn in a unit square with the following properties:

- The top-left and top-right corners are red points.
- The bottom-left and bottom-right corners are blue points.
- No three points are collinear.

Find an algorithm to draw red segments between red points and blue segments between blue points in such a way that: all the red points are connected to each other, all the blue points are connected to each other, and no two segments cross.

For example see below.



(a) Implement `draw_red_blue(R, B)` that takes the set of red points and blue points and returns a set of red segments (defined by 2 points) joining the red points and blue segments joining the blue points in $O(n \log n)$.

Note, given the vertices of a d -polygon, checking if a point is within the polygon is $O(d)$. In particular for a triangle or quadrilateral, it is $O(1)$. **(70 Marks)**

(b) Argue that your algorithm runs in $O(n \log n)$ time indeed. **(30 Marks)**

(c) Show the correctness of your algorithm. **(60 Marks)**